

Experiment No: 6

1. Aim:

To implement sentiment analysis using Natural Language Processing (NLP) techniques in Python and classify text data into positive, negative, or neutral categories.

2.Objectives:

1. Understand the concept of sentiment analysis and its applications.
2. Perform preprocessing of text data (tokenization, stopwords removal, stemming/lemmatization).
3. Implement feature extraction using Bag-of-Words and TF-IDF.
4. Train and evaluate a machine learning model (e.g., Naive Bayes, Logistic Regression) for sentiment classification.
5. Compare performance using different feature extraction techniques.

3.Procedure:

Step 1: Install required Python libraries : `pip install scikit-learn nltk`

Step 2: Import necessary modules (e.g., `CountVectorizer`, `TfidfVectorizer`, `MultinomialNB`, `train_test_split`, `classification_report`).

Step 3: Define or load a dataset containing text samples with sentiment labels (positive, negative, neutral).

☐ Example: Movie reviews, product reviews, or a small custom dataset.

Step 4: Preprocess the text data:

- ☐ Convert text to lowercase.
- ☐ Remove punctuation, numbers, and special characters.
- ☐ Remove stopwords (e.g., "is", "the", "and").
- ☐ Perform stemming/lemmatization if required.

Step 5: Split the dataset into training and testing sets.

Step 6: Extract features from the text using Bag-of-Words (BoW) representation.

☐ Convert text into numerical vectors using `CountVectorizer`. **Step 7:** Train a machine learning classifier (e.g., Naive Bayes) on the training set using BoW features.

Step 8: Test the trained model on the test set and evaluate performance using accuracy and classification report.

Step 9: Repeat feature extraction using TF-IDF representation.

- ☐ Convert text into TF-IDF vectors using TfidfVectorizer.

Step 10: Train the classifier again with TF-IDF features and evaluate model performance.

Step 11: Compare the results of Bag-of-Words vs TF-IDF to analyze differences in accuracy and classification.

Step 12: Display outputs such as:

- ☐ Preprocessed text samples.
- ☐ Feature matrices (BoW and TF-IDF).
- ☐ Model accuracy and classification reports.

Step 13: Conclude by highlighting which feature extraction method performs better for sentiment classification.

4. Question and answer:

1) What is sentiment analysis and why is it important?

Sentiment analysis (also called **opinion mining**) is a **Natural Language Processing (NLP)** technique used to determine the **emotional tone** behind a piece of text, speech, or data. It classifies opinions or statements as **positive, negative, neutral**, or sometimes more fine-grained emotions (e.g., happy, sad, angry, excited).

Importance :

- **Understands customer opinions** – Helps know how people feel about products, services, or brands.
- **Improves customer experience** – Identifies issues and areas where customers are satisfied or unhappy.
- **Supports decision-making** – Provides insights for product improvements and business strategies.
- **Monitors brand reputation** – Tracks public perception on social media and other platforms.
- **Saves time and effort** – Automates the analysis of large amounts of text data quickly.
- **Enhances marketing strategies** – Shows how customers react to campaigns and promotions.

2) How does preprocessing affect sentiment classification?

Preprocessing plays a very important role in sentiment classification because raw text usually contains noise (extra words, symbols, or irrelevant information). Proper preprocessing makes the data cleaner and more meaningful, which helps the model understand sentiments better.

Name:- Prathamesh Arvind Jadhav

Roll No:- 4059

Effects of preprocessing on sentiment classification:

1. **Removes noise:** Cleaning text (removing punctuation, numbers, or HTML tags) reduces irrelevant information.
2. **Handles stopwords:** Removing common words like *the*, *is*, *at* improves focus on important sentiment words.
3. **Normalization:** Converting text to lowercase ensures words like *Good* and *good* are treated the same.
4. **Stemming/Lemmatization:** Reduces words to their root form (e.g., *liked* → *like*), helping the model group similar meanings.
5. **Tokenization:** Splits text into words, making it easier for the model to analyze sentiment.
6. **Better accuracy:** With clean and standardized text, classifiers perform more effectively and give higher accuracy.

3) Why is Naive Bayes commonly used for text classification?

Naive Bayes is commonly used for text classification because it is simple, fast, and effective. It works well with high-dimensional data like text, where each word can be treated as a feature. Since it assumes features (words) are independent, the model is easy to build and requires less training data. It is also efficient in handling large datasets, gives good accuracy for problems like spam detection, sentiment analysis, and document categorization, and performs surprisingly well even with noisy or limited data.

4) Compare the results of BoW and TF-IDF in sentiment analysis.

Aspect	Bag of Words (BoW)	TF-IDF (Term Frequency–Inverse Document Frequency)
Representation	Counts the frequency of each word.	Considers word frequency and importance.
Weighting	Treats all words equally.	Gives higher weight to rare, meaningful words.
Effect of Common Words	Common words may dominate and add noise.	Reduces the weight of very common words.
Performance	Simple but may lower accuracy.	Usually improves accuracy in sentiment analysis.
Use Case	Best for small/simple text datasets.	Best for larger datasets and precise analysis.

5. Conclusion:

This experiment demonstrates how sentiment analysis can be implemented using NLP techniques. Bag-of-Words and TF-IDF provide different feature extraction methods, where TF-IDF generally performs better by reducing the weight of common words. Machine learning models such as Naive Bayes effectively classify text into sentiment categories (positive, negative, neutral).

AI&DS

NLP Lab

Name:- Prathamesh Arvind Jadhav

Roll No:- 4059